Session 2.3

The Loss, the Optimizer, and Three Samples

What the loss assumes, how the optimizer uses a gradient, and when to stop



© Zhigang Feng Redistribution prohibited without permission

Zhigang Feng

Session 2.3

The Loss, and the Optimizer

What the criterion assumes, and what to do with a gradient that is noisy



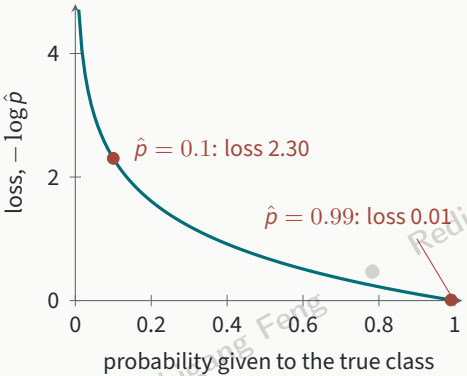
© Zhigang Feng Redistribution prohibited without permission

Loss I: Squared Error Is OLS

- For a continuous target, the mean squared error: $\mathcal{L} = \frac{1}{n}\sum_i (\hat{y}_i - y_i)^2$.
- Minimising it is maximum likelihood under Gaussian errors. For a linear f_θ it *is* ordinary least squares.
- A loss is therefore an assumption about the noise, not a convention. Change the noise model and the loss changes: Poisson likelihood for counts, the check function for quantiles.
- In Deck 2.4 the target is next month's yield. In Deck 2.5 the "prediction" is an Euler-equation residual and the target is zero, so no data enter the loss at all.
- **Squared error asks for the conditional mean. If that is not what you want, it is the wrong loss.**

© Zhiguo Feng. Redistribution prohibited without permission

Loss II: Cross-Entropy

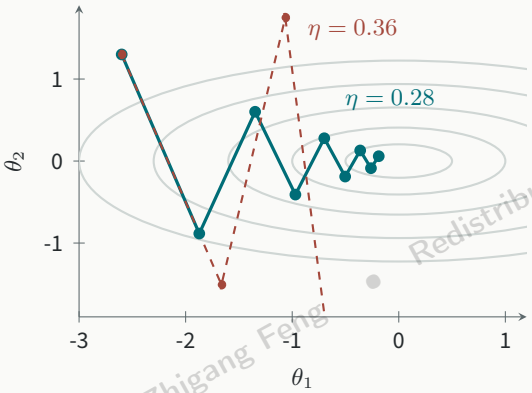


- A classifier returns a score z_k for each of K classes. The **softmax** turns scores into probabilities: $\hat{p}_k = e^{z_k} / \sum_j e^{z_j}$.
- **Cross-entropy** is the negative log-probability given to the true class, $\mathcal{L} = -\log \hat{p}_{k^\star}$.

© Zheng Feng
Redistribution prohibited without permission

Source: D. McFadden, "Conditional logit analysis of qualitative choice behavior", in *Frontiers in Econometrics*, 1974.

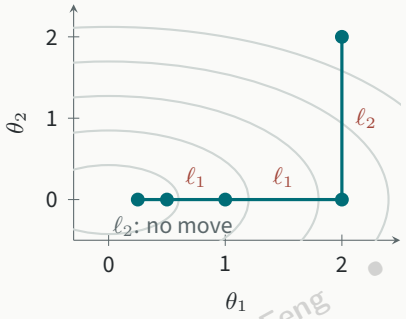
Gradient Descent: the Engine



© Zhigang Feng Redistribution prohibited without permission

- **Gradient descent**: $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}(\theta)$. Step against the gradient.
- The picture is $\mathcal{L} = \frac{1}{2}(\theta_1^2 + 6\theta_2^2)$. The teal path zigzags across the steep direction and converges.
- The red path uses a step only slightly longer. It overshoots further each time and diverges.
- **The learning rate is limited by the steepest direction, and progress by the flattest.**

Why a Sampled Gradient Still Gets There



- **Stochastic gradient descent** (SGD): differentiate a random subset of the loss's terms at each step.
- Here $\ell_1 = \frac{1}{2}\theta_1^2$, $\ell_2 = \theta_2^2$, step 0.5: only the drawn coordinate moves.
- Unbiased: its expectation is proportional to the full gradient.
- **The path is not monotone and is not meant to be — and the goal is not the lowest training loss but a minimum that generalises.**

Source: L. Bottou, F. Curtis, J. Nocedal, *SIAM Rev.* 60(2), 2018; A. Choromanska et al., AISTATS 2015; T. Garipov et al., NeurIPS 2018.

© Zhigang Feng Redistribution prohibited without permission

Batch, Stochastic, Mini-Batch

Method	Terms per step	Cost of a step	Path
full batch	all n	high	smooth
stochastic	1	lowest	very noisy
mini-batch	32 to 1,024	one matrix product	noisy, averaged

- An **epoch** is one pass through the training data; an **iteration** is one update. With 40,000 observations and batches of 256, an epoch is 156 full batches.
- Why mini-batch: many updates per pass, activations that fit in memory, hardware built for matrix products.
- The gradient noise also helps the iterate leave saddle points and shallow minima.

© Zhirang Feng · Redistribution prohibited without permission

Session 2.3

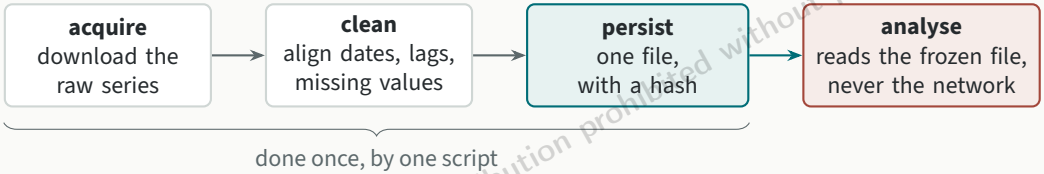
The Dataset, and Three Samples

Where the data come from, and the rule that makes a reported number mean something



© Zhigang Feng Redistribution prohibited without permission

The Data Workflow: Acquire, Clean, Persist — Once



© Zhiyang Feng · Redistribution prohibited without permission

- Statistical agencies revise. A notebook that downloads on every run gives different answers on different days.
- Separate the download from the analysis; the analysis then runs offline and gives the same numbers every time.
- **A result is reproducible only if its data are frozen.**

Train, Validation, Test: Three Jobs, Three Samples

training sample: choose the parameters θ	**validation**	**test**
the optimiser sees only this	choose the design: width, penalty, when to stop	report once, at the end

© Zheng Feng redistribution prohibited without permission

- A flexible model can fit its training sample as closely as you like. That says nothing about new data.
- **Validation** data choose between designs, so they are no longer a clean test.
- **Test** data are touched once. Their error estimates the error on future data.
- With time series the three samples are consecutive periods, never a shuffle.
- **Each sample does one job. A sample used for two jobs does neither honestly.**

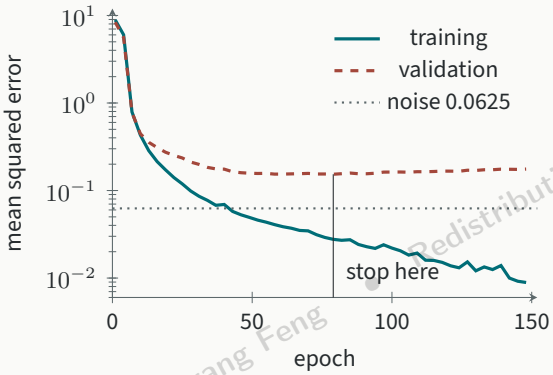
Leakage: Four Ways the Test Set Gets into Training

- **Leakage**: information from the test sample reaches the fitted model, so the test error is too good.
- **Shuffling a time series.** Neighbouring months are nearly identical; a shuffled split puts the answer for March next to the question for April.
- **Scaling with full-sample statistics.** The mean and standard deviation used to standardise the inputs must come from the training window alone.
- **Tuning on the test set.** Trying ten designs and reporting the best test error makes the test set a validation set.
- **The same template on both sides.** In the lab, sentences built as subject × phrase score 0.97 on a random split and 0.50 when whole phrases are held out.
- **Ask of every reported accuracy: what did the model already know about these cases?**

© Thiago Ferreira. All rights reserved. Redistribution prohibited without permission

Source: Accuracies: Session 2 lab notebook, appendix (logistic regression on word counts, 96 sentences).

Overfitting, Underfitting, and When to Stop

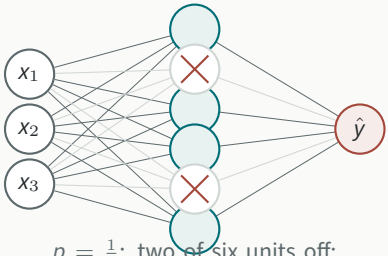


© Zhigang Feng Redistribution prohibited without permission

- **Underfitting**: both curves high. **Overfitting**: training loss falls while validation loss turns up.
- $n = 2{,}000$. Training loss ends at 0.0089, below the noise variance 0.0625: memorised noise.
- **Early stopping**: keep the best-validation parameters, near epoch 80.
- **A training loss below the noise floor is a diagnosis, not an achievement.**

Source: Recorded in the instructor's quantitative-macro course computations (2026); not recomputed in this session's lab.

Dropout, Step by Step



$p = \frac{1}{3}$: two of six units off; a new draw at every step

- **Dropout**: each step, each hidden unit is kept with probability $1 - p$, else zeroed: $\tilde{h} = m \odot h/(1-p)$, $m_j \sim \text{Bernoulli}(1-p)$, redrawn each step.
- The $1/(1-p)$ keeps $E[\tilde{h}] = h$; at prediction nothing is dropped and the full network runs once (`eval()` in Deck 2.4).
- $p = 0.1$ to 0.2 on hidden layers, never on the output.
- **A unit cannot count on any partner being present, so each must carry information on its own.**

© Zhigang Feng
Redistribution prohibited without permission

Source: N. Srivastava et al., "Dropout: a simple way to prevent neural networks from overfitting", *JMLR* 15, 2014.

Why Dropout Regularises

- With m hidden units there are 2^m thinned sub-networks; each step trains one of them, and all share one set of weights.
- At test time the full network approximates the average of those sub-networks: an ensemble for the price of one model.
- For a linear model, dropout on the inputs is a ridge penalty scaled by each regressor's second moment: an adaptive weight decay (Wager, Wang and Liang, 2013).
- It is noise on purpose: the network is trained to survive losing any one feature, as new data will make it.
- Cost: a noisier gradient and more steps; at $n = 50{,}000$ the next frame shows it is worth 1%.
- **Dropout pays where the data are too few to pin the weights down, and that is the economist's usual case.**

© Zhigang Feng · Redistribution prohibited without permission

Source: S. Wager, S. Wang, P. Liang, "Dropout training as adaptive regularization", *NeurIPS* 26, 2013; Srivastava et al. (2014).

Three Instruments Against Overfitting, and What They Are Worth

Strategy	best validation $n = 2{,}000$	best validation $n = 50{,}000$
none	0.1535	0.0788
weight decay 10^{-4}	0.1536	0.0804
dropout 0.1	0.1257	0.0780
dropout 0.2	0.1341	0.0792
both (0.1)	**0.1238**	0.0780

Noise floor 0.0625; 30 inputs, 17 pure noise.

- **Weight decay**: a penalty $\lambda\|\theta\|^2$, which is ridge regression.
- **Dropout**, from the previous two frames: a random share p of hidden units off at each step.
- Early stopping acts like a ridge penalty of about $1/(\eta T)$ after T steps.
- **Dropout pays where data are scarce: 18% at $n = 2{,}000$, 1% at $n = 50{,}000$. More data does more: 0.1535 to 0.0788.**

Source: Recorded in the instructor's quantitative-macro course computations (2026), median of three seeds; not recomputed in this session's lab.

© Zhigang Feng · Redistribution prohibited without permission

Takeaways, and What Is Still Missing

- **A loss is an assumption about the noise**: squared error is OLS under Gaussian errors; cross-entropy is the log-probability of the true class.
- **The optimizer steps against the gradient**: the learning rate is limited by the steepest direction, and a sampled gradient is unbiased, so mini-batches still get there.
- **Each sample does one job**: the optimiser sees the training data, the design is chosen on validation data, and the test data are touched once.
- **Generalisation is measured, not assumed**: a training loss below the noise floor is the mark of overfitting, and regularisation pays mainly when data are scarce. Deck 2.6 returns with a case the noise floor cannot diagnose at all.
- **Next, Deck 2.4: the same loop in PyTorch, first where the answer is known, then on next month's 10-year yield.**

© Lutang Peng · Redistribution prohibited without permission